EngageGraph: A Reproducible Benchmark and Toolkit for Temporal App-Engagement Prediction

Xinyi Cui
Cornell University
xc535@cornell.edu

November 2026

Abstract

App-engagement prediction is often demonstrated with a single notebook, a balanced AUC, and a pickled model. That combination is difficult to audit, easy to leak, and hard to operate. We present EngageGraph, an open-source benchmark and toolkit for *next-month new-app adoption* on panel-scale app-usage data. The protocol fits every component on January 2024, uses February only to construct binary adoption labels, ranks the full January catalogue with seen apps masked, and reports macro-averaged ranking metrics with paired user-level bootstrap intervals. We release four reference models—popularity, item-kNN, implicit ALS, and MetaPath2Vec—a pickle-free artefact format, a FastAPI service with cold-start handling and explanations, and a complete operational stack. On the January–February panel, item-kNN is the strongest reference model (NDCG@10 = 0.2685), while the graph embedding model underperforms despite its prominence in the original repository. We also audit the original notebook's reported 0.9257 AUC, identifying an elementwise cosine read-out, a February mapping mismatch, and a balanced-negative protocol that does not estimate deployment precision. The benchmark reports negative results and operational trade-offs alongside the best model. The engine is domain-agnostic: domain profiles adapt vocabulary, file layout, and serving blocklists, and a synthetic retail example exercises the same pipeline outside apps.

1 Introduction

Mobile app engagement is a sequential, sparse, and highly skewed phenomenon. A recommendation model that suggests the right app at the right time can improve retention; a model that only re-ranks the apps a user already has may look accurate while changing nothing. Distinguishing these outcomes requires a task definition, a temporal split, and a candidate set that match the decision being made.

The original `app-engagement` repository was a research upload: aggregated January and February 2024 panel exports, a MetaPath2Vec graph, model weights, and a notebook that produced app-switching probabilities. It was useful as an exploration, but it had three properties that make its headline numbers unsuitable for product decisions. First, February edges were used both to fit and to evaluate. Second, the evaluation sampled negatives by similarity, producing a balanced pair-classification task rather than a ranking task. Third, the cosine read-out averaged elementwise products, collapsing scores to a near-constant $1/d$ (0.0156 for $d = 64$), and the February embeddings were indexed with a different mapping than the one used to build them. The notebook's own output shows the symptoms: identical top-pair scores and zero overlap between January and February top pairs.

This paper makes four contributions:

1. A leakage-free temporal benchmark for next-month new-app adoption, with a fixed January catalogue, user eligibility rules, and explicit cold-start accounting.
2. A reproducible empirical comparison of popularity, item-kNN, implicit ALS, and MetaPath2Vec under one protocol, including coverage, novelty, and paired bootstrap comparisons.
3. An audit of the legacy AUC protocol that separates link-prediction discrimination from recommendation precision and documents the code-level defects.
4. An operable reference system: versioned artefacts, a typed Python package, a CLI, an HTTP API, a dashboard, Docker, CI, data/model cards, and runbooks.

We do not claim that item-kNN is the best possible model, nor that these results generalise beyond this panel and month pair. We claim that the protocol is auditable and that the reported trade-offs are real under it.

2 Related work

Collaborative filtering. Item-based CF [6] remains a strong baseline for sparse implicit data because it directly encodes co-usage. Implicit ALS [3] learns low-rank factors from confidence-weighted interactions, and BPR [5] optimises a pairwise ranking objective. EASE [8] is a closed-form item-item model that often outperforms deeper models on small catalogues; we leave it to future work.

Graph representation learning. metapath2vec [1] learns heterogeneous node embeddings from meta-path-guided random walks and a skip-gram objective with negative sampling. It is a natural fit for a user–app bipartite graph. Our result shows that a faithful implementation of the objective does not automatically produce a strong top-K recommender; the read-out and the ranking objective matter.

Reproducibility and documentation. Datasheets [2] and model cards [4] document provenance, intended use, and limitations. Hidden technical debt [7] describes why notebook-to-product transitions fail. EngageGraph treats these as first-class deliverables rather than appendices.

3 Data

The panel contains monthly aggregates of foreground app duration for January and February 2024. The raw exports have 34,024 eligible panelists, 9,848 catalogue apps, 2,077,959 January interactions, and 561,231 February adoption targets across 28,286 evaluation users. A further 10,568 February users have no January history and are reported as a cold-start slice; 307 more have January history but fewer than three catalogue items and are reported as ineligible. The weight column is inferred to be milliseconds from its distribution; all primary metrics are invariant to a global rescaling. The data card documents the unresolved provenance and the research-only licence.

The catalogue is fixed from January: apps used by at least ten distinct panelists, capped at the 10,000 most-supported apps. Users need at least three catalogue apps in January. Weights are clipped at the 99.9th percentile and transformed with $\log(1 + w)$. February is used only to label a user–app pair as an adoption if the app is in the January catalogue and was not used in January.

Table 1: Full-catalogue next-month adoption ranking on the January–February 2024 panel. Metrics are macro-averaged over evaluation users; the interval is a paired user-level bootstrap (1,000 resamples, 95%). Coverage and novelty make popularity concentration visible.

Model	NDCG@10 (95% CI)	Recall@10	MAP@10	HitRate@10	Coverage@10	Novelty@10
itemknn	0.2685 [0.2660, 0.2710]	0.1522	0.1516	0.8221	0.013	1.107
popularity	0.2351 [0.2327, 0.2376]	0.1347	0.1297	0.7723	0.006	1.055
metapath2vec	0.1639 [0.1622, 0.1660]	0.0933	0.0772	0.6979	0.015	1.661
als	0.0993 [0.0974, 0.1013]	0.0798	0.0505	0.4288	0.317	4.830

4 Protocol

For user u with January history H_u and February adoption set $R_u \subseteq I_{\text{Jan}} \setminus H_u$, every model produces a ranking of $I_{\text{Jan}} \setminus H_u$. The evaluator masks H_u centrally, so all models see the same candidate set. The primary metric is NDCG@10, macro-averaged over users; we also report Recall@10, MAP@10, HitRate@10, coverage@10, Gini@10, and novelty@10.

Uncertainty is estimated by a paired user-level bootstrap with 1,000 resamples. We report the 95% percentile interval for the mean difference against popularity and apply Holm–Bonferroni correction across the family of comparisons. The primary configurations (popularity, item-kNN $k = 100$, ALS) were fixed before the February evaluation. Two post-hoc diagnostics are disclosed in the repository: the MetaPath2Vec read-out was revised after its canonical dot product produced near-random top-10 rankings, and an item-kNN sweep was run as a sensitivity check while retaining the pre-specified $k = 100$. The MetaPath2Vec row is exploratory; the item-kNN vs. popularity result does not depend on either adjustment. The full protocol, including leakage controls, is in the repository's methodology document.

5 Models

Popularity scores each app by its January user support (log-scaled). It is a strong adjacent-month baseline and the cold-start fallback.

Item-kNN computes cosine similarity between item columns of the binary January matrix, retains the top 100 neighbours per item, and sums neighbour similarities over the user's history.

Implicit ALS uses confidence $c_{u,i} = 1 + \alpha r_{u,i}$ with $\alpha = 40$, 64 factors, $\lambda = 0.05$, and 15 sweeps [3].

MetaPath2Vec learns 64-dimensional embeddings from meta-path-guided walks on the bipartite graph with the objective of Dong et al. [1]. Because the jointly learned user vector is not optimised for top-K ranking, we score a user by the engagement-weighted mean of their January item embeddings, normalised and dotted with item embeddings. This read-out is documented as a design choice, not part of the original objective.

6 Results

Table 1 reports the primary comparison. Item-kNN achieves the best NDCG@10 (0.2685), improving on popularity with a paired interval that excludes zero (Table 2). The gain is modest in absolute terms but consistent: a Δ of roughly 0.03 NDCG@10 corresponds to moving relevant apps earlier in a 10-item slate for a substantial fraction of users.

Table 2: Paired comparisons against the popularity baseline on NDCG@10. Intervals are user-level paired bootstrap; p_{Holm} controls the family-wise error rate across candidates.

Candidate	Δ NDCG@10	95% CI	p_{Holm}
itemknn	+0.0334	[+0.0325, +0.0341]	0.0060
als	-0.1357	[-0.1380, -0.1335]	0.0060
metapath2vec	-0.0712	[-0.0731, -0.0692]	0.0060

ALS illustrates the difference between discrimination and top-K ranking. It separates February positives from random negatives well in a pairwise diagnostic, but its top-10 slate is dominated by niche items: NDCG@10 falls to 0.099 while coverage rises to 0.317. This is a classic implicit-feedback failure mode: reconstruction quality does not imply calibration for a popularity-skewed ranking metric.

MetaPath2Vec is weaker still under this protocol (NDCG@10 = 0.164 with profile scoring). A faithful implementation of a graph objective is not automatically a strong recommender, and the original repository's qualitative claims about app similarity did not constitute a ranking evaluation. We report it because omitting the model would weaken the comparison.

Coverage and novelty expose the trade-off that accuracy alone hides. Popularity and item-kNN concentrate almost all impressions on a few dozen apps (coverage@10 of 0.006 and 0.013), whereas ALS spreads recommendations across a third of the catalogue. A product team may prefer a blend; the benchmark makes the choice explicit instead of hiding it inside a single metric.

7 The legacy AUC

The original notebook reported ROC-AUC 0.9257 and precision 0.8552. Those numbers are reproducible in shape but not comparable to Table 1. The notebook trained and tested on February edges, sampled negatives by similarity, and selected the best of nine binary operators on the same labels. A balanced 1:1 test can show high AUC while precision at realistic prevalence is low; our audit reports both and computes positive predictive value at 1% and 5% prevalence using the test-set TPR and FPR.

Two code-level defects compound the problem. The "cosine" function returns an elementwise vector and the caller averages it, so the score is approximately $1/d$ rather than a dot product; the notebook's top pairs all print 0.0156. The February embedding matrix is filtered with indices from a different mapping, so top pairs mix unrelated apps. These are not subtle methodological preferences; they are implementation errors that make the reported top pairs arbitrary.

8 Operational design

The benchmark is packaged as a product, not a notebook. Artefacts contain a JSON manifest, NumPy arrays, mappings, and the January history; no pickle is used. The manifest records the dataset fingerprint, package version, git commit, model parameters, and evaluation metrics. The FastAPI service loads one artefact, masks seen items, returns explanations grounded in January co-usage, and falls back to popularity for unknown users with an explicit `cold_start_popularity` strategy. Health endpoints, request IDs, Prometheus counters, feedback capture, Docker, CI, and a retraining runbook are included.

The serving layer also exposes a within-user percentile $q_{u,i}$, the fraction of unseen candidates scored at or below item i. It is distribution-free and comparable across users with different history lengths; it is explicitly not a probability. The API accepts a minimum percentile, retaining only candidates at or above the corresponding empirical quantile; the returned percentile is still computed against the full unseen set. The formal definition and decision policy are in the repository's algorithm notes.

This engineering is not incidental. A model that cannot be rolled back, audited, or explained is not deployable, and a benchmark that cannot be rerun is not evidence.

The implementation is also domain-agnostic. The matrices, models, metrics, artefact format, and API operate on generic user–item interactions. A domain block supplies only vocabulary (for example, "product" instead of "app"), the optional serving blocklist, and display labels. The repository ships an app profile, a synthetic retail CSV profile, and an e-commerce template; the API reports these labels and the dashboard adapts automatically. The reference numbers in this paper remain app-panel-specific and should not be transferred to another domain without rerunning the protocol.

9 Limitations and ethics

The study covers one panel and one month pair. Panelists opt into measurement and may not represent the population. The weight unit is inferred, labels are binary, and panel attrition can look like disinterest. There are no demographics, so group fairness cannot be assessed; app usage can still reveal sensitive interests, and the data must not be used for re-identification or individual targeting. The code is Apache-2.0; the data is research-use only.

10 Reproducibility

The repository pins the environment, records source hashes, fixes seeds, and stores per-user metric arrays for paired comparisons. CI runs the test suite on synthetic data; the panel benchmark is reproduced locally or in a protected job. The evaluation command refuses to run if the artefact and dataset fingerprints differ, preventing a common silent failure. Known non-reproducibility (BLAS ordering, PyG walk order, vendor label changes) is documented rather than hidden.

11 Conclusion

EngageGraph turns a research notebook into an auditable benchmark and an operable recommendation service. Its most useful result may be negative: a graph embedding model that motivated the original repository does not beat a well-tuned item-kNN baseline on next-month app adoption. Reporting that result, alongside the protocol and the operational machinery, is the point.

References

[1] Yuxiao Dong, Nitesh V. Chawla, and Ananthram Swami. metapath2vec: Scalable representation learning for heterogeneous networks. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '17, pages 135–144, 2017. doi: 10.1145/3097983.3098036.

[2] Timnit Gebru, Jamie Morgenstern, Briana Vecchione, et al. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021. doi: 10.1145/3458723.

[3] Yifan Hu, Yehuda Koren, and Chris Volinsky. Collaborative filtering for implicit feedback datasets. In *Proceedings of the 8th IEEE International Conference on Data Mining*, ICDM '08, pages 263–272, 2008. doi: 10.1109/ICDM.2008.22.

[4] Margaret Mitchell, Simone Wu, Andrew Zaldivar, et al. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, FAT* '19, pages 220–229, 2019. doi: 10.1145/3287560.3287596.

[5] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. BPR: Bayesian personalized ranking from implicit feedback. In *Proceedings of the 25th Conference on Uncertainty in Artificial Intelligence*, UAI '09, pages 452–461, 2009.

[6] Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. Item-based collaborative filtering recommendation algorithms. In *Proceedings of the 10th International Conference on World Wide Web*, WWW '01, pages 285–295, 2001. doi: 10.1145/371920.372071.

[7] D. Sculley, Gary Holt, Daniel Golovin, et al. Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

[8] Harald Steck. Embarrassingly shallow autoencoders for sparse data. In *Proceedings of the World Wide Web Conference*, WWW '19, pages 3251–3257, 2019. doi: 10.1145/3308558.3313710.